

# 数据格式说明

每一组数据是一个文件夹，里面有三个纯文本文件：`map.txt`（地图）、`packages.txt`（包裹）、`car.txt`（小车）。你的程序必须能读入下面这个格式——我们评分时会用同样格式的数据喂给你的程序。

所有数值以空格 / 换行分隔，可用 C++ 的 `ifstream f; f >> x;` 逐个读入，非常简单。  
节点编号从 0 开始，0 号节点就是菜鸟驿站。

## 1. `map.txt`（校园地图，无向带权图）

```
n m
id x y is_station      ← 共 n 行，每行一个节点
u v w                  ← 共 m 行，每行一条无向边
```

| 字段                      | 含义                                                                 | 类型                                     |
|-------------------------|--------------------------------------------------------------------|----------------------------------------|
| <code>n</code>          | 节点数                                                                | 整数                                     |
| <code>m</code>          | 边数                                                                 | 整数                                     |
| <code>id</code>         | 节点编号， <code>0 .. n-1</code>                                        | 整数                                     |
| <code>x y</code>        | 节点坐标（仅供可选的可视化，算法用不到）                                               | 实数                                     |
| <code>is_station</code> | 是否驿站： <code>1</code> 是， <code>0</code> 否（只有 0 号为 1）                | 整数                                     |
| <code>u v w</code>      | 一条无向边：连接节点 <code>u</code> 、 <code>v</code> ，距离（边权）为 <code>w</code> | <code>u,v</code> 整数； <code>w</code> 实数 |

- 图保证连通，任意两点可达；边是无向的（`u→v` 与 `v→u` 距离相同）。
- 可能存在多条路径，最短路不一定是直连边。

## 2. `packages.txt`（包裹清单）

```
k
id weight dest arrive deadline      ← 共 k 行
```

| 字段                    | 含义                                 | 类型 |
|-----------------------|------------------------------------|----|
| <code>k</code>        | 包裹数                                | 整数 |
| <code>id</code>       | 包裹编号， <code>1 .. k</code>          | 整数 |
| <code>weight</code>   | 包裹重量 <code>w<sub>i</sub></code>    | 实数 |
| <code>dest</code>     | 目的地节点编号（不为 0）                      | 整数 |
| <code>arrive</code>   | 到站可派送时间 <code>s<sub>i</sub></code> | 实数 |
| <code>deadline</code> | 最晚送达时间 <code>t<sub>i</sub></code>  | 实数 |

## 3. `car.txt`（小车参数，仅一行）

speed car\_weight capacity

| 字段         | 含义                 | 类型 |
|------------|--------------------|----|
| speed      | 速度 $v$ （距离 / 单位时间） | 实数 |
| car_weight | 小车自重 $w_{car}$     | 实数 |
| capacity   | 装载重量上限 $w_{lim}$   | 实数 |

- 保证任何单个包裹的重量都不超过 `capacity`（即一定可行）。

## 4. 完整示例（对应 测试数据/示例/）

map.txt：

```
6 8
0 100 300 1
1 300 120 0
2 300 480 0
3 520 300 0
4 720 150 0
5 720 450 0
0 1 10
0 2 12
1 3 8
2 3 9
3 4 7
3 5 6
4 5 5
1 2 15
```

含义：6 个节点、8 条边；节点 0 是驿站；边 0-1 距离 10.....

packages.txt：

```
5
1 5 3 0 100
2 8 4 0 100
3 6 5 0 100
4 4 1 0 100
5 7 2 0 100
```

含义：5 个包裹；包裹 1 重 5、送到节点 3、0 时刻到站、时限 100.....

car.txt：

```
5 20 20
```

含义：速度 5、自重 20、容量上限 20。

该示例的最短路距离、成本公式算例，见 测试数据/示例/预期结果.md，可用来自检你的实现。

## 5. 关于你的程序的输出

---

输出格式由你自己设计（我们不做硬性规定），但必须清晰、可复现，并至少报告：

- **T1**：给定两点的最短距离与路径；
- **T2**：你的配送方案（每趟装哪些包裹、停靠顺序）与**不满意度之和**；
- **T3**：你的配送方案、**运送成本**、以及**超时包裹数**。

建议把这些结果打印到控制台或写入一个结果文件，方便你在报告和答辩里展示。